

실습강의안

2-1 Front-End 판교 5층 Day 1 / 4 난이도 보통 중요도 ★★★★★ 종합실습

Vue 기초와 반응형 시스템

Front-Framework: Vue.js · 2026-07-31 (금) · 09:00~18:00 (8H)

종합실습 이론 50% · 실습 50% 각 이론 뒤 종합실습 (Vue 시작하기·기본 문법)

🎯 학습 목표

Vue 3 프로젝트 구조와 SFC(Single File Component) 이해
반응형 상태와 템플릿 문법 활용

📖 핵심 개념

Vue

화면(UI)을 손쉽게 만들도록 도와주는 자바스크립트 프레임워크로, 데이터가 바뀌면 화면을 자동으로 다시 그려준다.

SFC(단일 파일 컴포넌트)

확장자가 .vue 인 파일 하나에 화면(template)-로직(script)-꾸밈(style)을 함께 담는 Vue의 기본 단위다.

ref

숫자·문자열 같은 값 하나를 반응형으로 감싸는 함수로, 자바스크립트에서 값을 쓸 때는 .value 를 붙인다.

computed(계산된 속성)

다른 반응형 값에서 자동 계산되는 값으로, 원본이 바뀔 때만 다시 계산되어 효율적이다.

Vite

Vue 프로젝트를 빠르게 시작하고 개발 서버를 띄워주는 빌드 도구로, 저장하면 화면이 즉시 갱신된다.

반응형(reactivity)

데이터(상태)가 바뀌면 그 데이터를 쓰는 화면이 저절로 따라 바뀌는 Vue의 **핵심** 동작이다.

디렉티브

v- 로 시작하는 HTML 특수 속성으로, v-if·v-for·v-bind 처럼 태그에 동작을 붙여준다.

watch(감시자)

특정 반응형 값이 바뀌는 순간을 지켜보다가, 바뀔 때마다 실행할 동작(부수 효과)을 걸어두는 함수다. computed가 새 값을 계산해 돌려주는 것이라면, watch는 값이 바뀌면 API를 부르거나 로그를 남기는 등 결과값이 아닌 행동이 필요할 때 쓴다.

세임네임 단축(same-name shorthand)

Vue 3.4부터 도입된 문법으로, 연결할 변수명과 HTML 속성명이 일치하면 `<img :src>`처럼 `= "src"` 부분을 생략할 수 있고 Vue가 같은 이름의 변수를 자동 매핑한다.
변수명을 `id`·`src`·`href`처럼 HTML 표준 속성명과 맞춰 두면 코딩이 빨라진다. (교재 p.61)

이벤트 수식어(event modifier)

이벤트 리스너의 기본 동작을 제어하는 특수 접미어다.
.prevent(폼 새로고침 방지)·.stop(버블링 차단)·.once(1회만)·.self 외에 .enter 같은 키보드 수식어, .ctrl·.exact 시스템 수식어, .right 마우스 수식어까지 있다. (교재 pp.79-82)

스캐폴딩(scaffolding)

npm create vue@latest 실행 시 폴더 구조·빌드 설정·필수 라이브러리 연동을 자동 생성해 주는 초기 뼈대 구축이다.
생성 과정에서 Router·Pinia·ESLint·Prettier 탑재 여부를 질문으로 선택한다. (교재 p.25)

v-model 수식어(Modifiers) — .number · .trim · .lazy

v-model 뒤에 점을 찍어 붙이는 편의 기능(Syntactic Sugar)이다. 입력 요소의 동작 방식이나 수집되는 데이터 형태를 바꾼다. .number 는 입력값을 Number 타입으로 자동 형변환하고(안 붙이면 숫자를 쳐도 문자열로 들어온다), .trim 은 앞뒤 공백을 잘라내며, .lazy 는 input 대신 change 시점에 반영한다. 수식어는 필요한 만큼 이어 붙일 수 있다(Modifier Chaining).

심화 이론

computed vs watch, 언제 무엇을 쓰나

computed는 '다른 값들로부터 자동으로 만들어지는 새 값'에 씁니다.

예를 들어 장바구니 합계처럼, 원본이 바뀌면 알아서 다시 계산되어 화면에 그대로 보여줄 값에 적합합니다.

watch는 '값이 바뀌었을 때 무언가를 실행'해야 할 때 씁니다. 검색어가 바뀌면 서버에 재검색을 요청하거나, 입력을 로컬스토리지에 저장하는 일이 그렇습니다.

판단 기준은 한 줄로 정리됩니다 — 새 값이 필요하면 computed, 바뀔 때 할 일이 필요하면 watch.

왜 Vue 같은 프레임워크를 쓰나요?

옛날 방식에서는 화면의 글자 하나를 바꾸려면 document.getElementById 로 요소를 찾아 직접 글자를 갈아끼워야 했습니다.

데이터가 많아지면 '어디를 바꿔야 하는지' 챙기는 일이 금세 복잡해집니다.

Vue는 이 일을 대신 해 줍니다.

우리는 그냥 '데이터'만 바꾸면, Vue가 그 데이터를 쓰는 화면 부분을 알아서 다시 그려 줍니다.

마치 엑셀에서 셀 값을 바꾸면 그 셀을 참조하는 합계가 자동으로 바뀌는 것과 같습니다.

그래서 개발자는 '화면 조작'이 아니라 '데이터와 규칙'에만 집중할 수 있습니다.

반응형 상태: ref 와 reactive

Vue에서 화면과 연결되는 데이터를 '상태(state)'라고 부릅니다.

상태를 반응형으로 만들려면 값을 ref 나 reactive 로 감싸야 합니다.

ref 는 숫자·문자열·불리언 같은 '하나짜리 값'을 감쌀 때 씁니다.

감싸고 나면 자바스크립트 코드에서는 .value 로 접근하지만, 템플릿(화면) 안에서는 .value 없이 바로 씁니다.

reactive 는 여러 값을 담은 '객체'를 통째로 감쌀 때 쓰며 .value 가 필요 없습니다.

초보자는 우선 'ref 만 쓴다'고 외워도 충분합니다.

교재 결론(p.97): reactive는 객체를 통째로 교체하거나 구조분해하면 반응성이 끊기는 약점이 있어, 현업에서는 객체·배열도 안전하게 ref()로 통일해 쓰는 추세가 강하다.

2교시 — Vite로 시작하는 이유와 개발환경 한 장 정리

강의 흐름: 먼저 '왜 손으로 index.html·script 태그를 만들지 않고 Vite를 쓰나'를 묻습니다. 옛날에는 자바스크립트 파일을 여러 개 만들고 순서대로 script 태그로 끼워 넣어야 했고, 라이브러리 하나 추가할 때마다 CDN 주소를 붙였습니다. Vite는 이 모든 잡일을 대신 해 주는 '개발 도구 세트'입니다.

핵심 포인트 세 가지로 설명합니다. 첫째, 스캐폴딩 — 'npm create vite'를 한 번 치면 표준 폴더 구조(src, public, index.html, vite.config.js)가 자동으로 만들어져 어느 회사 프로젝트를 열어도 구조가 같습니다. 둘째, 개발 서버(npm run dev) — 코드를 저장하는 순간 브라우저가 새로고침 없이 바뀐 부분만 갈아끼웁니다. 이것을 HMR(Hot Module Replacement)이라 부르고, 입력하던 값이 그대로 남아 있어 개발이 빨라집니다. 셋째, 빌드(npm run build) — 배포할 때는 이 도구가 코드를 압축해 dist 폴더를 만들어 줍니다.

왜 중요한가: 프론트엔드 개발의 8할은 '저장→화면 확인'의 반복입니다. HMR이 이 순환을 1초 이내로 줄여 주므로, 학습자가 직접 HMR을 체감해 보는 것이 이 교시의 목표입니다. WSL/Node.js 위에서 Vite가 도는 구조(Node.js=자바스크립트 실행 엔진, Vite=그 위의 개발 도구)를 그림 한 장으로 정리해 주면 됩니다.

3교시 — SFC(단일 파일 컴포넌트) 한 파일에 화면·로직·꾸밈

강의 흐름: .vue 파일 하나를 열어 보여 주면서 시작합니다. 이 파일은 세 칸으로 나뉩니다. template(화면에 보이는 HTML), script setup(동작을 정하는 자바스크립트), style(꾸밈 CSS). 예전에는 화면은 .html, 로직은 .js, 꾸밈은 .css로 파일이 흩어져 있어서 버튼 하나를 고치려면 세 파일을 오가야 했습니다. SFC는 '한 기능=한 파일'로 묶어 관련된 것을 한눈에 봅니다.

핵심 포인트: template 안에는 반드시 화면 요소를 적고, {{ }} 이중 중괄호로 script의 값을 꽂아 넣습니다. script setup은 Vue 3의 최신 문법으로, 별도 return 없이 선언한 변수·함수가 template에서 바로 쓰입니다. style에 scoped를 붙이면 그 꾸밈이 이 컴포넌트 안에만 적용됩니다. main.js가 App.vue를 읽어 index.html의 #app 자리에 붙이는 것이 앱이 뜨는 원리라는 점도 짚어 줍니다.

왜 중요한가: 앞으로 만드는 모든 화면이 결국 이 .vue 파일들의 조합입니다. '한 파일 안 세 칸의 역할'을 손에 익히면 어떤 컴포넌트를 열어도 어디를 봐야 하는지 바로 압니다. 기본으로 생성된 App.vue를 지우고 빈 SFC에 h1 하나를 직접 적어 화면에 띄우는 것을 함께 해 봅니다.

5교시 — 템플릿 디렉티브 v-if·v-for·v-bind로 화면을 데이터에 연결

강의 흐름: 앞에서 만든 반응형 상태를 '화면에 어떻게 반영하나'가 이 교시의 주제입니다. 디렉티브는 v-로 시작하는 특수 속성으로, HTML 태그에 '동작'을 붙이는 리모컨이라고 설명합니다. 세 가지만 확실히 잡습니다.

핵심 포인트: v-if는 조건이 참일 때만 그 요소를 화면에 그립니다(거짓이면 아예 없어짐). v-else와 짝지어 '데이터가 있으면 목록, 없으면 안내 문구'처럼 분기합니다. v-for는 배열을 하나씩 꺼내 같은 모양의 요소를 반복해 찍어 냅니다. 이때 :key에 항목마다 다른 고유값(보통 id)을 주어야 Vue가 어떤 항목이 바뀌었는지 정확히 추적한다는 점을 꼭 강조합니다(key를 인덱스로 대충 주면 목록 편집 시 버그가 납니다). v-bind는 콜론(:) 한 글자로 줄여 쓰며, :src::class처럼 HTML 속성값을 데이터와 연결합니다. :class에 객체({ done: todo.done })를 주면 조건에 따라 클래스가 붙었다 떨어졌다 합니다.

왜 중요한가: '데이터 → 화면'의 세 가지 기본기(조건·반복·속성연결)로 실무 화면의 대부분을 만듭니다. 과일 배열을 v-for로 찍고, 비었을 때 v-if로 안내 문구를 띄우는 것을 라이브 코딩으로 보여 줍니다.

6교시 — 이벤트 v-on(@)과 폼 양방향 바인딩 v-model

강의 흐름: 5교시가 '데이터를 화면에 보여주기'였다면, 이 교시는 반대로 '사용자 입력을 데이터로 받아오기'입니다. 두 개의 디렉티브로 완성됩니다.

핵심 포인트: v-on은 골뱅이(@)로 줄여 쓰며 클릭·엔터 같은 사용자 사건에 함수를 연결합니다. @click='addTodo'는 '클릭하면 addTodo를 실행하라'는 뜻이고, @keyup.enter처럼 점(.)으로 키를 특정할 수 있습니다(수식어라고 부름). v-model은 입력창과 상태를 '양방향'으로 묶습니다. 즉 사용자가 타이핑하면 상태가 바뀌고, 코드가 상태를 바꾸면 입력창 글자도 바뀝니다. 이 양방향성이 v-bind(한 방향, 데이터→화면)와의 결정적 차이라는 점을 대비해서 설명합니다. 체크박스에 v-model을 걸면 true/false가, select에 걸면 선택값이 자동으로 상태에 담깁니다.

왜 중요한가: 회원가입·검색·할 일 추가 등 모든 입력 화면의 뼈대가 v-model 하나입니다. 입력창에 v-model='name'을 걸고 바로 아래 {{ name }}을 두어, 타이핑이 즉시 반영되는 '거울 효과'를 눈으로 확인시키는 것이 이 교시의 **핵심** 실습입니다. 8교시 미니 실습(카운터·할 일 목록)은 오늘 배운 ref+디렉티브+이벤트+v-model을 한 화면에 모으는 종합 연습이라는 점을 예고하며 마무리합니다.

MVVM 아키텍처 — 화면과 데이터 사이의 중개자 (교재 pp.6-9)

Vue.js는 MVVM(Model-View-ViewModel) 아키텍처 패턴을 따른다. Model은 컴포넌트 내부의 순수한 데이터와 비즈니스 로직(ref:reactive 변수), View는 사용자에게 실제로 보이는 DOM과 template 영역, ViewModel은 둘 사이를 잇는 중재자로 Vue 프레임워크 자체가 이 역할을 한다. 식당에 비유하면 주방(Model)과 손님 테이블(View) 사이를 오가는 종업원(ViewModel)과 같다 — 주방 음식이 바뀌면 테이블에 새로 내오고, 손님 주문이 바뀌면 주방에 전달한다.

Vue는 DOM Listeners와 Data Bindings 메커니즘으로 이 중개를 자동화하며, 그 대표 장치가 v-model의 양방향 데이터 바인딩이다. 덕분에 UI를 그리는 부분과 데이터 처리 로직이 완전히 분리된다.

Virtual DOM — 진짜 DOM이 느린 이유와 두 가지 최적화 (교재 p.8)

브라우저의 **Real DOM** 조작은 느리다 — JavaScript로 실제 DOM을 수정하면 브라우저는 Layout(Reflow)과 화면을 다시 색칠하는 Paint(Repaint)를 매번 거친다.

교재의 예처럼 3,000개 노드 중 단 1개만 수정해도 3,000개를 새로 그리면 엄청난 연산 부하가 생긴다.

Virtual DOM은 이 비용을 줄이려고 메모리 위에 만든 가짜 DOM으로, 학급 게시판 통째로 새로 만드는 대신 바뀐 종이 한 장만 떼어 붙이는 것과 같다.

공간적 최적화(바뀐 노드만 Layout-Paint)와 시간적 최적화(한 이벤트 안에 여러 번 데이터가 바뀌어도 끝난 시점에 단 한 번만 실제 DOM 반영), 두 방향으로 렌더링 성능을 끌어올린다.

v-html과 XSS — 편리한 만큼 위험한 디렉티브 (교재 pp.51-53)

v-html은 변수에 담긴 문자열을 실제 HTML 요소로 해석해 화면에 주입하는 디렉티브로, 내부적으로 `element.innerHTML`과 동일하게 동작한다.

문제는 XSS(Cross-Site Scripting) 공격 노출이다 — 해커가 게시판 댓글에 악성 자바스크립트를 심어두면, 다른 사용자가 그 글을 읽을 때 그 사용자의 브라우저에서 해커의 코드가 실행되어 쿠키·세션 토큰을 탈취당한다.

대문에 아무나 붙여 놓은 쪽지를 명령서로 믿고 그대로 실행하는 집과 같다.

교재는 `img` 태그의 `onerror` 속성에 이동 코드를 심으면 다른 사이트로 강제 이동되는 실험으로 이 위험을 직접 보여주며, 디렉티브 총람표에도 v-html에 "보안사고(XSS) 유의"를 명시한다.

사용자 입력을 v-html로 그대로 출력하는 일은 피해야 한다.

📖 상세 학습 내용

Vue 프로젝트 시작

- `npm create vite` 로 프로젝트 생성
- `npm run dev` 로 개발 서버 실행
- `src/App.vue` 진입점 이해
- `main.js` 가 앱을 화면에 붙이는 과정

반응형 상태

- `ref` 로 단일 값 감싸기
- `reactive` 로 객체 감싸기
- 자바스크립트에선 `.value`, 템플릿에선 그대로
- 상태가 바뀌면 화면 자동 갱신

템플릿 문법

- `{{ }}` 보간으로 값 출력
- `v-bind(:)`로 속성 연결
- `v-on(@)`으로 이벤트 연결
- `v-model` 로 폼 양방향 바인딩

개발환경과 Vue Devtools

- Node.js/WSL 위에서 Vite 개발 서버 구동
- 브라우저 Vue Devtools 확장 설치
- Devtools의 Components 탭에서 각 컴포넌트의 `ref/props` 실시간 값 관찰
- 상태를 바꿨을 때 트리에서 값이 갱신되는지 눈으로 확인

🕒 진행 시간표 (과목 확정 시간표 · 교시 길이 가변)

1교시

09:00-09:50

1. Vue.js 시작하기 — 프레임워크가 필요한 이유·Vite 프로젝트·SFC 구조 (개념)

이론

2교시

10:00-10:50

종합실습 — 개발환경 구성·프로젝트 생성 및 실행·HMR 체감 (종합실습)

종합실습

3교시

11:00-11:40

2. Vue 기본 문법 (1) — 템플릿 문법과 반응형 상태 ref-reactive (실습)

실습

4교시

11:50-12:20

2. Vue 기본 문법 (2-1) — 조건과 반복 v-if·v-for (실습)

실습

12:20-13:40

점심 휴식

5교시

13:40-14:20

2. Vue 기본 문법 (2-2) — v-if·v-for 마무리 (실습)

실습

6교시

14:30-15:20

2. Vue 기본 문법 (3) — 속성과 이벤트 v-bind·v-on (실습)

실습

7교시

15:30-16:20

2. Vue 기본 문법 (4) — 폼 바인딩 v-model과 수식어 (실습)

종합실습

8교시

16:30-17:50

종합실습 — 반응형 UI·카운터·할 일 목록 완성 (종합실습)

종합실습

 실습

종합과제 1단계 — 날씨 Mockup (교재 2장 [실습] 과제)

1. 정본 소스 저장소 <https://github.com/bottletiger/skala-vue> 를 Fork 하거나 Clone 한다. 본인 계정 저장소 이름은 skala-vue 로 맞춘다.
2. 임의의 날씨 데이터 배열을 만든다 — `weatherList = ref([ { id: 'city_01', name: '서울', temp: 28, status: '맑음' }, { id: 'city_02', name: '수원', temp: 24, status: '비' }, { id: 'city_03', name: '부산', temp: 26, status: '구름' } ])`
3. v-for 로 날씨 카드를 반복 출력한다. :key 에 id 를 바인딩하는 것은 필수다 — 인덱스를 쓰지 않는다.
4. v-if 로 라벨을 붙인다. 기온 25도 이상이면 '🔥 더움 (25도 이상)', 25도 미만이면 '❄️ 선선했 (25도 미만)'.
5. :value 와 @input 으로 검색 입력을 양방향 처리한다. 한글 입력(IME 조합)이 깨지지 않는지 직접 타이핑해 확인한다.
6. 여기까지가 1일차 결과물이다. 커밋하고 push 한다.

 산출물 · 날씨 카드 목록이 반복 출력되고, 기온에 따라 라벨이 갈리며, 검색어 입력이 한글로도 동작하는 화면

 추가 실습 (Lab)

Lab 1. 첫 Vue 프로젝트 띄우기

1. 터미널을 열고 'npm create vite@latest hello -- --template vue' 를 입력한다
2. 'cd hello && npm install && npm run dev' 를 차례로 실행한다
3. 브라우저에서 <http://localhost:5173> 에 접속해 기본 화면을 확인한다
4. src/App.vue 의 <template> 안 글자를 바꿔 저장하고 화면이 즉시 바뀌는지 본다

Lab 2. 반응형 이름 인사 만들기

1. App.vue 의 <script setup> 에 'import { ref } from "vue"' 를 적는다
2. const name = ref('') 로 이름 상태를 만든다
3. <template> 에 <input v-model="name"> 를 두고 그 아래 <p>안녕하세요, {{ name }}님</p> 을 적는다
4. 입력창에 글자를 칠 때마다 인사 문구가 실시간으로 바뀌는지 확인한다

Lab 3. Vue Devtools로 반응형 상태 들여다보기

1. 브라우저에 Vue Devtools 확장을 설치한다
2. 개발 서버를 띄우고 Devtools의 Components 탭을 연다
3. 입력창에 글자를 칠 때마다 todos/text 상태가 트리에서 바뀌는 것을 관찰한다

 실습 예제

ref 로 만든 반응형 카운터 (vue)

```

1  <script setup>
2  // 반응형 값을 만드는 ref 함수를 가져온다
3  import { ref } from 'vue'
4  // 숫자 0 을 반응형으로 감싼 카운터 상태를 만든다
5  const count = ref(0)
6  // 버튼을 누르면 카운터를 1 증가시키는 함수(자바스크립트에서는 .value 필요)
7  function plus() { count.value++ }
8  </script>
9
10 <template>
11   <!-- 현재 카운트를 화면에 출력한다(템플릿에서는 .value 없이 바로 사용) -->
12   <p>현재 값: {{ count }}</p>
13   <!-- 버튼 클릭 시 plus 실행, 누를 때마다 위 숫자가 1씩 증가 -->
14   <button @click="plus">+1</button>
15 </template>

```

 복사

💡 버튼을 누르면 count.value 만 바뀌도 화면 숫자가 자동으로 올라간다.

v-if 와 v-for 디렉티브 (vue)

```

1  <script setup>
2  // 반응형 배열을 만들기 위해 ref 를 가져온다
3  import { ref } from 'vue'
4  // 과일 이름 3개를 담은 반응형 배열
5  const fruits = ref(['사과', '바나나', '포도'])
6  </script>
7
8  <template>
9   <!-- 배열이 비어 있지 않을 때만(v-if 조건 참) 목록을 보여준다 -->
10   <ul v-if="fruits.length > 0">
11     <!-- v-for 로 과일을 하나씩 꺼내 li 로 출력, key 로 인덱스 i 사용 -->
12     <li v-for="(f, i) in fruits" :key="i">{{ i + 1 }}. {{ f }}</li>
13   </ul>
14   <!-- 위 조건이 거짓이면(배열이 비면) 이 문구를 대신 보여준다 -->
15   <p v-else>과일이 없습니다.</p>
16 </template>

```

 복사

💡 v-for 는 반복 출력을, v-if/v-else 는 조건에 따른 화면 분기를 담당한다.

이벤트 v-on과 v-model로 만드는 실시간 입력 거울 (vue)


```

1  <script setup>
2  // 반응형 값을 만드는 ref 함수를 가져온다
3  import { ref } from 'vue'
4  // 입력창과 연결할 이름 상태(처음엔 빈 문자열)
5  const name = ref('')
6  // 인사 횟수를 세는 상태(버튼 클릭마다 1씩 증가)
7  const count = ref(0)
8  // 버튼을 눌렀을 때 실행: 인사 횟수를 1 올린다
9  function greet() {
10     count.value++ // 자바스크립트에서는 .value 로 값에 접근
11 }
12 </script>
13
14 <template>
15   <!-- v-model 로 입력창과 name 을 양방향 연결(타이핑하면 아래 문구가 즉시 바뀜) -->
16   <input v-model="name" placeholder="이름을 입력하세요" />
17   <!-- name 이 비어 있지 않을 때만 인사 문구를 보여준다 -->
18   <p v-if="name">안녕하세요, {{ name }}님!</p>
19   <!-- 비어 있으면 안내 문구를 대신 보여준다 -->
20   <p v-else>이름을 입력해 주세요.</p>
21   <!-- @click 으로 버튼 클릭에 greet 함수를 연결 -->
22   <button @click="greet">인사하기</button>
23   <!-- 클릭할 때마다 늘어나는 인사 횟수를 표시 -->
24   <p>지금까지 {{ count }}번 인사했어요.</p>
25 </template>

```

 복사

💡 v-model 은 입력→데이터를 자동으로 채우는 양방향 연결이고, @click(v-on)은 사용자 사건에 함수를 거는 단방향 연결이다. 타이핑이 곧바로 화면에 비치는 '거울 효과'와 클릭이 상태를 바꾸는 흐름을 한 화면에서 함께 체감하게 하는 데모다.

watch 로 부수효과 처리 + 디바운스 (javascript)

```

1  import { ref, watch } from 'vue'
2
3  const keyword = ref('')
4  let timer
5  // keyword가 바뀔 때마다 실행되지만, 300ms 디바운스로 API 호출을 줄인다
6  watch(keyword, (val) => {
7     clearTimeout(timer) // 이전 예약 취소
8     timer = setTimeout(() => {
9        fetchSuggestions(val) // 실제 네트워크 호출
10     }, 300) // 입력이 멈춘 뒤에만 호출
11 })

```

 복사

💡 computed(파생 값) vs watch(부수효과)의 차이를 이해한다. 검색 입력은 디바운스로 호출 폭주를 막는다.

v-bind 클래스 바인딩 — 상태에 따라 스타일 켜고 끄기 (vue)



```

1  <script setup>
2  // 반응형 값을 만드는 ref 를 가져온다
3  import { ref } from 'vue'
4  // 활성화 여부(참이면 active 클래스가 붙음)
5  const isActive = ref(true)
6  // 에러 여부(참이면 빨간 경고 스타일)
7  const hasError = ref(false)
8  </script>
9
10 <template>
11   <!-- 객체 구문: 값이 true 인 클래스만 실제로 적용된다 -->
12   <p :class="{ active: isActive, danger: hasError }">상태에 따라 색이 바뀝니다</p>
13   <!-- 배열 구문: 기본 클래스 + 삼항으로 상황별 클래스를 함께 건다 -->
14   <p :class="['box', isActive ? 'on' : 'off']">항상 box, 그리고 on/off 토글</p>
15   <!-- 버튼으로 상태를 뒤집어 실시간 변화를 확인 -->
16   <button @click="isActive = !isActive">active 토글</button>
17   <button @click="hasError = !hasError">error 토글</button>
18 </template>
19
20 <style scoped>
21 .active { color: blue; font-weight: bold; } /* active 붙으면 파란색 */
22 .danger { background: #ffd6d6; }           /* danger 붙으면 빨간 배경 */
23 .box { padding: 8px; }
24 </style>

```

💡 :class 객체 구문은 조건이 true 인 클래스만 붙이고, 배열 구문은 기본 클래스와 조건부 클래스를 함께 건다. CSS 를 직접 토글하지 않고 데이터로 스타일을 제어하는 것이 **핵심**이다.

폼 요소마다 다른 v-model — 체크박스·라디오·셀렉트 (vue)



```

1  <script setup>
2  import { ref } from 'vue'           // 반응형 변수 도구
3  const agree = ref(false)           // 단일 체크박스 → true/false
4  const fruits = ref([])             // 여러 체크박스 → 고른 value 가 배열로 쌓임
5  const gender = ref('')             // 라디오 → 고른 값 하나
6  const city = ref('seoul')          // 셀렉트 → 선택한 option 값
7  </script>
8
9  <template>
10   <!-- 단일 체크박스: 담기는 값은 true/false -->
11   <label><input type="checkbox" v-model="agree" /> 약관 동의</label>
12
13   <!-- 같은 배열에 묶인 체크박스: 체크한 value 들이 배열에 모인다 -->
14   <label><input type="checkbox" value="사과" v-model="fruits" /> 사과</label>
15   <label><input type="checkbox" value="바나나" v-model="fruits" /> 바나나</label>
16
17   <!-- 라디오: 같은 v-model 을 공유하면 하나만 선택된다 -->
18   <label><input type="radio" value="남" v-model="gender" /> 남</label>
19   <label><input type="radio" value="여" v-model="gender" /> 여</label>
20
21   <!-- 셀렉트: 고른 option 의 value 가 city 에 들어간다 -->
22   <select v-model="city">
23     <option value="seoul">서울</option>
24     <option value="busan">부산</option>
25   </select>
26
27   <!-- 선택 결과를 실시간 확인 -->
28   <p>동의:{{ agree }} / 과일:{{ fruits }} / 성별:{{ gender }} / 도시:{{ city }}</p>
29 </template>

```

💡 같은 v-model 이라도 요소에 따라 담기는 값이 다르다 — 단일 체크박스는 불리언, 여러 체크박스는 배열, 라디오·셀렉트는 고른 값 하나다.

이벤트 수식어 — .prevent 로 기본동작 막고 .stop 으로 전파 막기 (vue)



```

1  <script setup>
2  import { ref } from 'vue'
3  const log = ref('아직 클릭 전') // 무슨 일이 일어났는지 기록
4  function goLink() { log.value = '페이지 이동을 막고 함수만 실행됨' }
5  function outer() { log.value = '바깥 상자 클릭' }
6  function inner() { log.value = '안쪽 버튼만 클릭(바깥으로 안 번짐)' }
7  </script>
8
9  <template>
10   <!-- .prevent : a 태그의 기본 페이지 이동을 취소하고 함수만 실행 -->
11   <a href="https://naver.com" @click.prevent="goLink">이동 대신 함수 실행</a>
12
13   <!-- 바깥 div 클릭 시 outer 실행 -->
14   <div @click="outer" style="padding:20px; background:#eee">
15     바깥 영역
16     <!-- .stop : 클릭이 바깥 div 까지 번지는 것(버블링)을 차단 -->
17     <button @click.stop="inner">나만 반응하는 버튼</button>
18   </div>
19
20   <p>{{ log }}</p>
21 </template>

```

💡 .prevent 는 링크·폼 제출 같은 브라우저 기본동작을 막고, .stop 은 안쪽 클릭이 바깥 요소까지 전파되는 것을 막는다. 수식어를 붙이면 함수 안에서 preventDefault·stopPropagation 을 직접 부를 필요가 없다.

reactive() 로 객체 상태 묶어 관리하기 — 재할당 금지 주의 (vue)



```

1  <script setup>
2  // 객체·배열을 통째로 반응형으로 만드는 reactive 를 가져온다
3  import { reactive } from 'vue'
4  // 이름과 나이를 한 덩어리로 묶은 반응형 객체 (스크립트에서 .value 불필요)
5  const user = reactive({ name: '이순신', age: 30 })
6  // 버튼 클릭 시 나이만 1 올리는 함수
7  function birthday() {
8    user.age++ // 알맹이 속성만 바꾸면 화면이 자동으로 갱신된다
9  }
10 // (주의) user = { name: '홍길동', age: 0 } 처럼 통째로 재할당하면
11 // 반응형 연결이 끊어져 화면이 더 이상 갱신되지 않는다!
12 </script>
13
14 <template>
15   <!-- reactive 객체는 템플릿에서도 점 표기로 바로 읽는다 -->
16   <p>이름: {{ user.name }} / 나이: {{ user.age }}세</p>
17   <!-- 클릭할 때마다 나이가 한 살씩 늘어난다 -->
18   <button @click="birthday">나이 한 살 추가</button>
19 </template>

```

💡 ref 는 어떤 값이든 감싸고 .value 로 접근하지만, reactive 는 객체 전용이고 .value 없이 쓴다. 대신 새 객체로 통째로 재할당하면 반응성이 끊기는 약점이 있어, 현업에서는 객체도 그냥 ref 로 통일하는 추세가 강하다.

v-if vs v-show — 없애서 숨기기 vs 가려서 숨기기 (vue)

```

1  <script setup>
2  import { ref } from 'vue'    // 반응형 값 도구
3  const open = ref(true)      // 열림/닫힘 상태(두 상자가 공유)
4  </script>
5
6  <template>
7    <!-- 클릭할 때마다 참/거짓이 뒤집힌다 -->
8    <button @click="open = !open">토글 (현재: {{ open }})</button>
9
10   <!-- v-if: 거짓이면 태그 자체를 DOM 에서 없애 버린다(개발자도구에서 사라짐) -->
11   <p v-if="open">v-if 상자 - 조건이 거짓이면 아예 존재하지 않음</p>
12
13   <!-- v-show: 태그는 그대로 두고 CSS display:none 으로 눈에만 숨긴다 -->
14   <p v-show="open">v-show 상자 - 숨겨져도 DOM 에는 남아 있음</p>
15 </template>

```

 복사

💡 v-if 는 처음에 거짓이면 아예 그리지 않아 초기 비용이 낮지만, 바뀔 때마다 태그를 부수고 다시 지어 전환 비용이 크다. 그래서 모달·탭처럼 전환이 잦은 곳은 v-show, 로그인 후 화면처럼 전환이 드문 곳은 v-if 를 쓴다. v-show 는 v-else 조합이 안 된다는 점도 기억하자.

toRefs — reactive 객체를 구조분해해도 반응성 지키기 (vue)

```

1  <script setup>
2  // reactive 객체와, 속성을 반응형 그대로 꺼내는 toRefs 를 가져온다
3  import { reactive, toRefs } from 'vue'
4  // 도시와 기온을 묶은 반응형 객체
5  const state = reactive({ city: '수원', temp: 24 })
6  // (X) const { city, temp } = state → 일반 값 복사라 반응성이 끊긴다
7  // (O) toRefs 로 감싸면 모든 속성이 반응형 ref 로 추출된다
8  const { city, temp } = toRefs(state)
9  // 꺼낸 뒤에는 ref 이므로 스크립트에서 .value 로 다룬다
10 function warmer() { temp.value++ } // 원본 state.temp 도 함께 바뀐다
11 </script>
12
13 <template>
14   <!-- 추출한 ref 를 바꿔도 원본 객체와 같은 값이 유지된다 -->
15   <p>{{ city }} 기온: {{ temp }}도 (원본 확인: {{ state.temp }}도)</p>
16   <button @click="warmer">기온 +1</button>
17 </template>

```

 복사

💡 reactive 객체를 그냥 구조분해하면 반응형 연결이 끊어지는 것이 대표적인 초보 실수다. toRefs 는 모든 속성을, toRef 는 속성 1개만 반응형 ref 로 추출해 이 문제를 해결한다.

watch 로 reactive 객체 감시 — 통째 감시 vs 특정 속성 조준 (vue)



```

1  <script setup>
2  import { reactive, ref, watch } from 'vue'
3  // 상품 정보를 묶은 reactive 객체
4  const state = reactive({ name: '노트북', price: 1000 })
5  const log = ref('아직 변동 없음') // 감시 결과를 보여줄 문구
6
7  // 방법1) 객체를 통째로 감시: 내부 어떤 속성이 바뀌어도 실행되지만
8  // 새값과 옛값이 같은 객체를 가리켜 '진짜 이전 값'을 알 수 없다
9  watch(state, () => { console.log('내부 어딘가가 바뀜(이전 값 추적 불가)') })
10
11 // 방법2) () => state.price 처럼 화살표 함수로 특정 속성만 조준하면
12 // 이전 값이 제대로 보존되어 변화 폭까지 계산할 수 있다
13 watch(() => state.price, (newP, oldP) => {
14   log.value = oldP + '원 → ' + newP + '원 (' + (newP - oldP) + '원 변동)'
15 })
16 </script>
17
18 <template>
19   <p>{{ state.name }}: {{ state.price }}원</p>
20   <!-- 버튼을 누르면 두 감시자가 모두 반응한다 -->
21   <button @click="state.price += 500">가격 +500</button>
22   <p>{{ log }}</p>
23 </template>

```

💡 reactive 객체는 변수명을 그대로 넣으면 자동으로 깊은 감시가 되지만 이전 값을 못 쓴다. 객체 안의 특정 속성 변화를 낚아채고 싶으면 **반드시** 화살표 함수(getter) 형태로 조준해야 옛값·새값 비교가 가능하다.

🔧 실전 소스

App.vue — 반응형 할 일 목록 전체 (vue)



```

1  <script setup>
2  // vue 라이브러리에서 반응형 함수 ref 와 계산 속성 computed 를 가져온다(화면과 데이터를 연결하기 위함)
3  import { ref, computed } from 'vue'
4
5  // 할 일 객체들을 담을 배열을 반응형으로 만든다(배열이 바뀌면 화면도 자동 갱신)
6  const todos = ref([])
7  // 입력창에 적은 글자를 담을 반응형 문자열(처음엔 빈 문자열)
8  const text = ref('')
9  // 새 할 일의 고유 번호를 만들기 위한 카운터(1부터 시작)
10 let nextId = 1
11
12 // 버튼을 눌렀을 때 실행될 함수: 새 할 일을 목록에 추가한다
13 function addTodo() {
14   // 입력값 앞뒤 공백을 제거했을 때 비어 있으면 아무것도 하지 않고 함수를 끝낸다
15   if (text.value.trim() === '') return
16   // todos 배열 끝에 새 할 일 객체를 추가한다(id·제목·완료여부)
17   todos.value.push({ id: nextId++, title: text.value, done: false })
18   // 다음 입력을 위해 입력창 값을 다시 비운다
19   text.value = ''
20 }
21
22 // 아직 완료되지 않은 할 일의 개수를 자동 계산한다(todos 가 바뀔 때만 다시 계산)
23 const remaining = computed(() => todos.value.filter(t => !t.done).length)
24 </script>
25
26 <template>
27   <!-- 화면 제목과 남은 할 일 개수를 보여준다(remaining 값이 바뀌면 숫자도 자동 변경) -->
28   <h2>할 일 목록 (남은 일: {{ remaining }}개)</h2>
29   <!-- 입력창: v-model 로 text 상태와 양방향 연결, 엔터를 누르면 addTodo 실행 -->
30   <input v-model="text" @keyup.enter="addTodo" placeholder="할 일을 입력하세요" />
31   <!-- 추가 버튼: 클릭하면 addTodo 함수를 호출한다 -->
32   <button @click="addTodo">추가</button>
33   <!-- 할 일들을 세로 목록으로 표시한다 -->
34   <ul>
35     <!-- v-for 로 todos 배열을 하나씩 꺼내 li 를 만든다, key 에는 고유 id 를 준다 -->
36     <li v-for="todo in todos" :key="todo.id">
37       <!-- 체크박스: v-model 로 해당 할 일의 done (완료여부) 과 양방향 연결 -->
38       <input type="checkbox" v-model="todo.done" />
39       <!-- 완료되면 done 클래스가 붙어 취소선이 그려진다, 제목을 출력 -->
40       <span :class="{ done: todo.done }">{{ todo.title }}</span>
41     </li>
42   </ul>
43 </template>
44
45 <style>
46 /* done 클래스가 붙은 글자에 취소선과 회색을 적용한다(완료 표시) */
47 .done { text-decoration: line-through; color: gray; }
48 </style>

```



ref 로 만든 상태를 v-model-v-for-computed 로 연결한 완결형 예제다.

입력→추가→체크가 모두 데이터만 바꾸면 화면이 자동으로 따라 바뀐다.

검색어를 감시해 재검색하는 watch 예제 (vue)

```

1  <script setup>
2  // ref로 검색어 상태를, watch로 그 변화를 감시한다
3  import { ref, watch } from 'vue'
4
5  // 입력창과 연결될 검색어 상태(처음엔 빈 문자열)
6  const keyword = ref('')
7  // 검색 결과를 담은 반응형 배열
8  const results = ref([])
9
10 // keyword가 바뀔 때마다 실행: 새 값(newVal)을 받아 서버에 재검색을 요청한다
11 watch(keyword, async (newVal) => {
12   if (!newVal.trim()) { results.value = []; return } // 빈 검색어면 결과를 비우고 종료
13   const res = await fetch('/api/search?q=' + newVal) // 실제로는 검색 API를 호출
14   results.value = await res.json() // 받은 결과로 화면을 갱신
15 })
16 </script>
17
18 <template>
19   <!-- v-model로 입력창과 keyword를 양방향 연결 -->
20   <input v-model='keyword' placeholder='검색어를 입력하세요' />
21   <!-- 검색 결과를 목록으로 표시 -->
22   <ul><li v-for='r in results' :key='r.id'>{{ r.title }}</li></ul>
23 </template>

```

복사



computed는 값 계산에, watch는 값이 바뀔 때의 '부수 효과'에 쓴다.

검색어가 바뀔 때 서버를 다시 부르는 것처럼 무언가를 실행해야 하는 일은 watch로 처리한다.



과제

1. 오늘 만든 할 일 목록에 '완료된 항목 모두 삭제' 버튼을 추가하고, 클릭 시 done 이 true 인 항목만 배열에서 제거되도록 구현해 캡처를 제출한다
2. computed 를 하나 더 만들어 '전체 개수 / 완료 개수' 를 화면에 함께 표시한다
3. **교재 2장 과제(p.91)** — 날씨 제어판 UI-카드 모형: v-for 카드(:key 필수) + v-if 온도 라벨 + :value/@input 검색 + @click.stop 상세보기